Protection Needs Elicitation (PNE) & CIA Mapping

SecureShare · Team Codex · CYC386 Midterm Lab Exam (Fall 2026)

Team: Zain Ali (FA23-BCT-059), Nimra Umar (FA23-BCT-031) **Application:** SecureShare — hardened file sharing & collaboration service **CDF reference:** Unit 2 (Protection Needs Elicitation, CIA triad)

1. Purpose

This document identifies the **assets** of SecureShare, elicits **protection needs** from each stakeholder perspective, maps every need explicitly to the **CIA triad** (Confidentiality, Integrity, Availability), and derives **measurable security requirements**. Each requirement is traced to a Jira issue and, downstream, to a STRIDE threat (see `THREAT_MODEL.md`) and a remediation commit/PR.

---

2. Asset Inventory

Assets are classified as **Data (D)**, **Process (P)**, **Component (C)** and **External Entity (E)**.

ID	Asset	Type	Description	Sensitivity
A1	User credentials	D	Usernames + Argon2id password hashes	Critical
A2	Session cookies / tokens	D	Authenticated session identifiers	Critical
A3	Uploaded files (content)	D	User documents stored on disk	High
A4	File metadata	D	Owner, original name, size, timestamps	Medium
A5	Share grants	D	Which user may access which file	High
A6	User database	C	SQLite store (users/files/shares)	Critical
A7	Authentication process	P	Login, session issuance, logout	Critical
A8	Authorization process	P	Object-level access decisions	Critical
A9	File export / preview process	P	subprocess + native C helper	High
A10	Native preview helper	C	`thumbgen_secure` shared library	High
A11	CI/CD pipeline & secrets	C	GitHub Actions, `SECRET_KEY`, tokens	High
A12	Web UI (rendered pages)	C	Dashboard, auth, share console	Medium
E1	Registered user	E	Owns and shares files	—
E2	Administrator	E	Operates the service	—
E3	Anonymous visitor / attacker	E	Untrusted external entity	—

---

3. Stakeholders & Perspectives

Stakeholder	Primary concerns
User	My files stay private; only people I pick can see them; the service is up; nobody can act as me.
Administrator / **Operator**	The platform is available, tamper-evident, and cannot be abused to run commands on the server.
Attacker (adversarial)	Wants to read others' files (IDOR), run commands (injection/overflow), forge actions (CSRF), trick users (clickjacking), or take the service down.
Instructor / **Assessor**	Needs traceable evidence that each need is met and verified.

---

4. Elicited Protection Needs → CIA Mapping

Each need is mapped to one or more of **C**onfidentiality, **I**ntegrity, **A**vailability.

PN ID	Protection Need (stakeholder)	Asset(s)	C	I	A
PN-01	Passwords must never be stored or exposed in recoverable form	A1, A6	✔	✔	
PN-02	A user's files must be readable only by the owner and explicit grantees	A3, A5, A8	✔		
PN-03	No user may access another user's object by guessing/altering an ID (IDOR)	A3, A4, A8	✔	✔	
PN-04	User-supplied input must never be interpreted as a server command	A9, A10	✔	✔	✔
PN-05	Memory-unsafe components must not be exploitable via crafted input	A9, A10	✔	✔	✔
PN-06	State-changing actions must originate from the genuine user (anti-CSRF)	A7, A3, A5		✔	
PN-07	Sensitive pages must not be framed by a malicious site (clickjacking)	A12, A7		✔	
PN-08	Sessions must resist hijacking, fixation and theft	A2, A7	✔	✔	
PN-09	The service must remain available under abuse (brute force, large uploads)	A6, A7, A9			✔
PN-10	Secrets/credentials must not leak via code, logs or the repo	A11, A1	✔		
PN-11	Uploaded content must not be stored/served in a way that enables path traversal or script execution	A3, A12	✔	✔	
PN-12	Security regressions must be caught before reaching `main`	A11 (all)	✔	✔	✔

CIA coverage summary: Confidentiality 9/12 · Integrity 8/12 · Availability 4/12.

---

5. Derived Measurable Security Requirements

Each requirement is testable/automatable and traced to a Jira issue.

SR ID	Requirement (measurable)	Satisfies	Verification	Jira
SR-01	All passwords hashed with Argon2id; no plaintext in DB/ logs	PN-01, PN-10	`test_password_is_hashed_not_plaintext`	SCRUM-11
SR-02	Object-level authorization on **every** file read/ mutation	PN-02, PN-03	`test_idor_other_user_is_forbidden` (403)	SCRUM-14
SR-03	No shell interpretation of user-controlled strings; argv + allow-list	PN-04	`test_command_injection.py`	SCRUM-15
SR-04	Native copies bound by buffer capacity, not caller length; hardened flags	PN-05	ASAN clean on secure build; `test_make_preview_is_bounds_safe`	SCRUM-12
SR-05	CSRF synchronizer token required on all state-changing POSTs	PN-06	`test_delete_without_csrf_token_is_rejected` (400)	SCRUM-16
SR-06	`X-Frame-Options: DENY` + CSP `frame-ancestors 'none'` on all responses	PN-07	`test_clickjacking.py`	SCRUM-17
SR-07	Session cookies `HttpOnly` + `SameSite=Lax` + `Secure` (HTTPS); strong protection	PN-08	header inspection; config review	SCRUM-18

SR-08	Rate limiting on auth + upload; 10 MB upload cap	PN-09	config review; `@limiter.limit`	SCRUM-19
SR-09	`SECRET_KEY`/DB URL from environment; `.env` git-ignored	PN-10	repo scan; `.gitignore`	SCRUM-20
SR-10	Upload extension allow-list + server-generated storage names (`secure_filename`)	PN-11	upload route review	SCRUM-21
SR-11	CI fails build on high/critical SAST findings in production code	PN-12	Bandit `-ll` gate; Semgrep `--error`	SCRUM-22

---

6. Traceability (PNE → Threat → Fix → Test)

The complete chain is maintained on the Jira board (`docs/jira/board_export.md`). Example rows:

PNE	STRIDE threat	CWE	Fix (commit/PR)	Test
PN-03	Information Disclosure / Elevation	CWE-639	`fix/idor` → PR #3	`test_idor.py`
PN-04	Tampering / Elevation	CWE-78	`fix/command-injection` → PR #4	`test_command_injection.py`
PN-05	Tampering / DoS / Elevation	CWE-120	`fix/buffer-overflow` → PR #2	ASAN + `test_auth_and_overflow.py`
PN-06	Tampering / Spoofing	CWE-352	`fix/csrf` → PR #5	`test_csrf.py`
PN-07	Tampering (UI redress)	CWE-1021	`fix/clickjacking` → PR #6	`test_clickjacking.py`

---

7. Residual Needs / Assumptions

- HTTPS/TLS termination is assumed at a reverse proxy in production; `Secure` cookies and HSTS are enabled accordingly (`COOKIE_SECURE=1`).
- SQLite is adequate for the lab; a production deployment would move to a managed RDBMS with encryption at rest (extends PN-01/PN-02).
- Rate-limit storage is in-memory for the lab; production would use Redis.

These residual items are carried into the risk register (`THREAT_MODEL.md §6`).